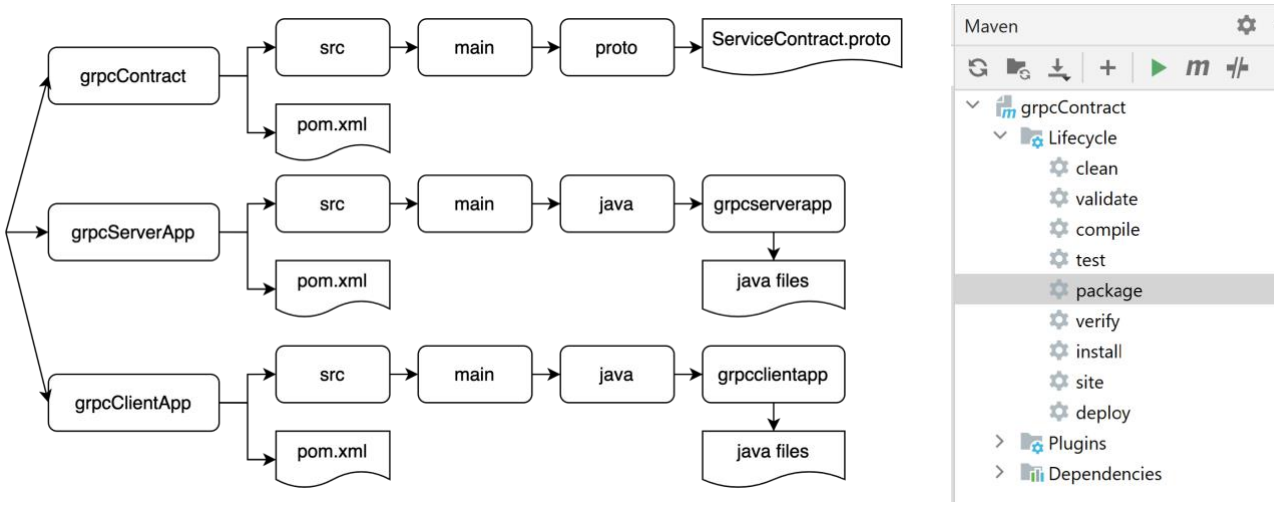
Lab 2

Objectives:

- Develop distributed services powered by *Google Remote Procedure Call* (gRPC)
- Distinguish between unary calls and calls with *server* stream

Part 1

1) Consider the example of the three gRPC projects presented and demonstrated in the lesson (appendix `grpc-Slides-Example.zip`), with the structure presented in Figure 1a). The `grpcServerApp`, `grpcClientApp`, and `grpcContract` projects have the Maven directory structure with their pom.xml. The `grpcContract` project has the contract defined in the `NumbersServiceContract.proto` file which, by convention of the `protobuf` compiler, is in the `\src\main\proto directory`. See below (attached section) how to open Maven projects in IntelliJ.



a) b)

Fig. 1: a) Structure of directories and files of the projects provided; b) Maven Window in IntelliJ

In the project `grpcContract`, using the Maven menu in the IntelliJ environment, manages the JAR with the contract (action ***package***) and install it in the local Maven repository (action ***install***).

2) Compile the source code of the server and client and manage their JAR using the Maven package option. Verify that there are no compilation errors and test the server and client by trying out the different operations available in the contract. You can run both applications directly in IntelliJ and through the command line, using the JAR obtained earlier.

3) If the server does not implement the `isAlive operation` of the contract, will the operation still be available on *the client's* stub?

4) Add a new operation to find which prime numbers exist in a given range:

   a) Add to the contract the following server-stream operation, the purpose of which is to get the prime numbers that exist in the range [start; end]:

```
rpc findPrimes(IntervalNumbers) returns (stream IntNumber)
message IntervalNumbers { int32 start = 1; int32 end = 2; }
```

   b) Implement the new contract operation on the server.

   c) Develop a new client application that, using a non-blocking *stub*, makes 5 calls to the `findPrimes` operation to calculate the prime numbers between 1 and 500, in intervals of 100 numbers ([1; 100], [101; 200], …). Note that because calls are made with a *non-blocking stub*, the next call is made without the previous call having already been completed (i.e., without waiting for all the prime numbers from the previous call to be received). Also, for this reason, responses will arrive in parallel to the client, although it is not necessary to explicitly manage *Threads* on either the client side or the server side.

   d) After testing on your local machine, create the server project artifact (*package* option in the Maven menu), copy the artifact (JAR with all dependencies) to a VM in your GCP project, and test the client and server applications in this *deployment scenario*.

Part 2

1) Consider a system for exchanging messages (*Forum*) in which sending and receiving messages is organized into topics. The contract for the service to be made available by the server application is presented below:

```
syntax = "proto3";
import "google/protobuf/empty.proto";
option java_multiple_files = true;
option java_package = "forum";
package forum;

service Forum {
   subscribe a topic
   rpc topicSubscribe(SubscribeUnSubscribe) returns (stream ForumMessage);
   unsubscribe a topic
   rpc topicUnSubscribe(SubscribeUnSubscribe) returns (google.protobuf.Empty);
   get all topics in server
   rpc getAllTopics(google.protobuf.Empty) returns (ExistingTopics);
   send a message to a topic
   rpc publishMessage(ForumMessage) returns (google.protobuf.Empty);
}

message SubscribeUnSubscribe {
   string usrName = 1;
   string topicName = 2;
}
message ExistingTopics {
   list of topic names
   repeated string topicName = 1;
}
message ForumMessage {
   string fromUser = 1;
   string topicName = 2;
   string txtMsg = 3;
}
```

The contract represents the following features:

- `topicSubscribe`: Operation that receives a user's unique identifier and the name of the topic they want to subscribe to (e.g. sports, economy, politics). From that moment on, the user receives the messages sent to that topic. A new topic is created if there is no one with the name indicated yet.
- `topicUnSubscribe`: Operation that receives the user identifier and the name of the topic you want to unsubscribe from.
- `publishMessage`: Operation that receives the user's name, the name of the topic, and the text message to be published to the topic. It is only allowed to publish messages in a topic that has already been subscribed. If the user has not subscribed to the topic, error information should be sent to the client.
- `getAllTopics`: Operation that returns a list with the names of all discussion topics that exist on the server.

a) Deploy the server application that provides the *Forum* service. It is suggested that server state management for connected topics and users be supported by the following data structure:

```
map: { (topic, map: { (username, streamObserver) } ) }
ConcurrentMap<String, ConcurrentMap<String, StreamObserver<ForumMessage>>>
```

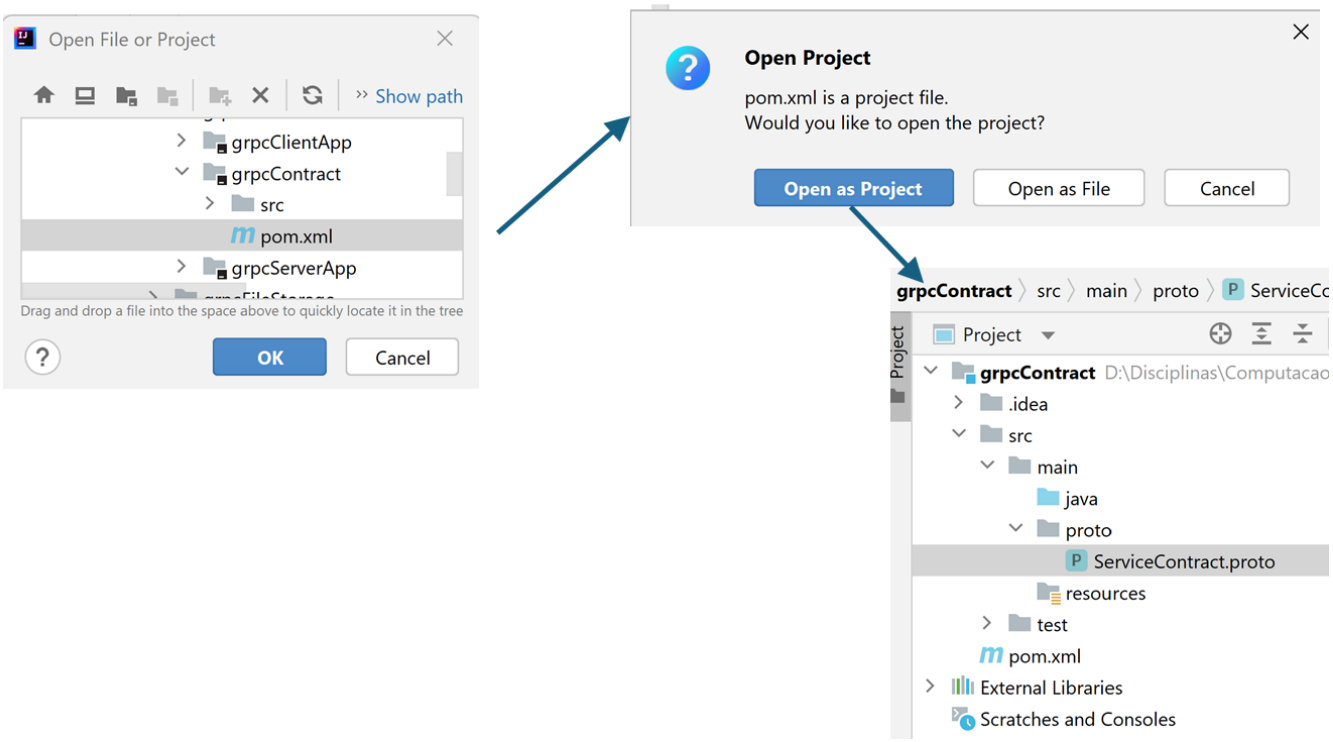
This structure consists of a *hash map* whose key is the name of a topic and whose value is another *hash map* whose key is the name of the users, who subscribed to the topic, and the value is its *stream observer* where a user will receive the messages of the topic he subscribed to.

b) Deploy a console-mode client application that allows users to exchange messages across multiple topics.

c) Test the system by running the server on a GCP VM and clients on the personal computer of each member of the group.

d) *Checkpoint*: In the second lesson dedicated to this lab, you should provide the IP address of the VM where the server is running so that the teacher can use a client application to access the server of each group. Register the public IP of the VM in this document: https://docs.google.com/spreadsheets/d/1p_6yeY6nskkqqCuTtIOCGiT7p3armoCMSuuWWG4ByPE/edit?usp=sharing

ANNEX

How to open projects in *IntelliJ* from the Maven (*pom.xml*) description.

Abrir o projeto Maven selecionando na estrutura de diretorias o ficheiro *pom.xml*